COMP2008 – Mobile App Development

Group Assessment

Group Project Planner:

Design and Implementation of a Task Allocation and Deadline Tracking App

Part A Report

Group members:

Aliyah Richshaine Macawile (22072366)

Twisha Domadia (22875086)

Manasviba Chauhan (23127979)

Semester 2, 2026

Submission date: [DD Month 2026]

1. Problem and Users

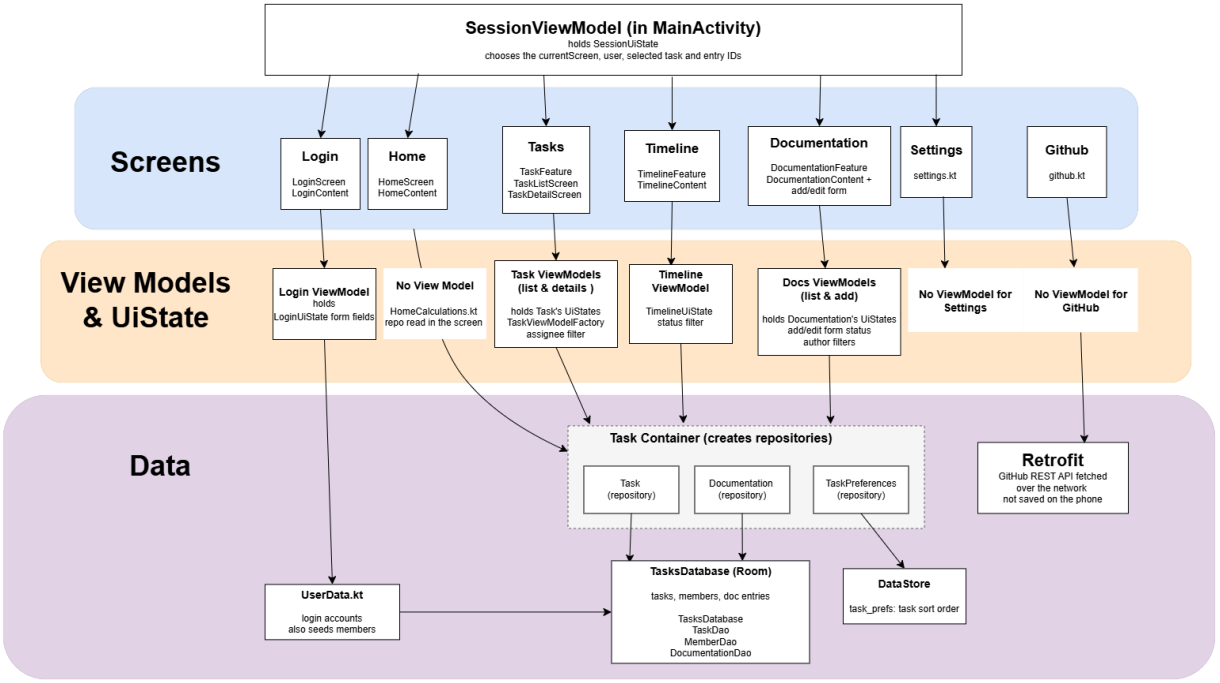
Our users are small student project teams, working on group assignments that require coordinating tasks, deadlines, and shared documentation across a semester. At present, these teams manage this work across a patchwork of disconnected tools: a group chat for quick updates, a shared document or spreadsheet to track who is doing what, and GitHub for the code itself. None of these tools link ownership, deadlines and progress together in one place, so it becomes easy to lose track of which task belongs to which member, when it is actually due, and how much of the project is genuinely finished.

The app's core task is to let a student project team allocate tasks to individual members and track those tasks against the project's key deadlines, all in one place. A member creates a task, assigns it to themselves or a teammate, sets a priority using MoSCoW (Must/Should/Could/Won't), and can see at a glance how many days remain or how overdue it is. The Home screen surfaces each member's own completion percentage and their next few open tasks, colour coded by status, so progress is visible without opening the full list.

The app's data centres on three related entities. Members are local login accounts (username and hashed password) stored in Room. Tasks are each assigned to one member and hold a title, description, start and due date, status, and MoSCoW priority. Documentation entries are dated notes and screenshots linked to timeline items. One member can own many tasks, which keeps ownership and progress meaningfully connected rather than incidental.

Two features are explicitly out of scope. Real-time, end-to-end encrypted group chat was excluded because it would need a backend server and multi-device synchronisation to keep messages consistent, outside this unit's local-data focus. Account creation was also excluded: registering a new account securely would need additional validation and recovery handling beyond what this assignment covers. The app instead logs in against a small set of existing local accounts, which is sufficient to demonstrate job allocation and personalised views without that added risk.

2. Architecture



Note: for better view of the Diagram refer to Documentation/MAD-Architecture.png

2.1. Where State Lives

The state lives in four places, depending on how long it must persist. Tasks, members and documentation entities are saved in Room Database (TasksDatabase) and the task sort order is stored in a DataStore, so both survive closing the app. The repositories expose the Room data as a Flow, so an edit in Tasks appears in the Timeline and Documentation screens without a manual refresh.

ViewModels hold a state that survives rotation but not when closing the app. The list screens (Tasks, Timeline and Documentation) keep only the user's filter choice, such as the assignee filter and derive everything else from the database. The form screens (Login, task and documentation) keep what the user has typed until Save, so rotating the device does not discard it.

MainActivity holds the app-wide settings (dark mode, font size and the GitHub owner and repository), so the theme and font scale apply to every screen. It uses rememberSaveable to survive rotation, but it's not stored and resets when the app is closed.

Composables hold only transient interface states, such open menus and scroll positions, which mostly resets on rotation. The exceptions are the task form's date picker and the Timeline's open task sheet, which use rememberSaveable.

3. Design Decisions

3.1 Github

One design decision that was made during development was to allow users to view GitHub repository activity directly inside the application rather than being redirected to an external browser. This was implemented using Retrofit, GitHub REST API and Jetpack Compose. Even though our focus was to implement the activity directly inside the application we still decided to allow the option to redirect to the github repository. This was done using implicit intent which allows the user to open the GitHub repository to an external browser. While this was simpler to implement and we could have left it there, we were determined to integrate the activity directly, as it would require users to leave the application whenever they wanted to view the commits, integrating it within the application allows for a more consistent and convenient experience and reduces unnecessary navigation. Retrofit was used to send HTTP requests to GitHub's REST API and retrieve repository commit information which includes; commit messages, authors and dates. This was then displayed within the application using Jetpack Compose, which allows users to view commits through the Github screen.

Another advantage of this approach is the ability to control how the commits are presented. Rather than having the whole website, we can retrieve and display specific information, in this case commits. A LazyColumn was used to display these commits in a vertically scrollable list. We could've used Column and verticalScroll() instead, which composes all its child components. However, LazyColumn uses a lazy composition to handle larger lists more efficiently. Some repositories may be from big projects with a lot of commits so LazyColumn was the most suitable for displaying repository activity, where the number of commits may increase over time.

Although this was implemented to make the user experience more convenient, a limitation in the current program is that the information is displayed as a basic list with very minimal visual organisation. As the repository grows users will find it difficult to locate specific commits and distinguish between the activity. Future improvements could include categorising them by colour, like in the timeline and tasks, improving the visual layout to make it so much easier to navigate. With our implementation of GitHub, any visual changes would be simple to do as we have full control of how the information is displayed.

3.2 Timeline

Another design decision was to implement a Gantt chart style timeline which visually represents the project tasks, durations and overall progress. The timeline uses horizontal bars to represent individual tasks, allowing users to understand when the tasks are scheduled and how they link to the project's timeline. This was chosen because Gantt charts are often needed as a visual aid in group projects to represent project schedules, which makes it easier for users to track activities and deadlines.

An important consideration was accessibility, particularly for those with colour vision deficiencies. Although colour is a useful way to distinguish task statuses (to-do, in progress, done and overdue), relying on colour alone can make the app difficult for some users, such as those with colour-blind deficiency. We've thought of adding clear text labels and indicators to every element, however we found that this is not always practical on a small screen, so we chose a colour palette that stays distinguishable across four types of colour blindness. With that in mind, we used Adobe's colour blindness simulator (Adobe, n.d.) to check if the colours are safe. In such cases that there are users with other variants of colour deficiencies, they can go to the tasks bar and view their status there.

Another key choice was to integrate the gantt chart directly with the documentation's feature. Users can select a specific bar to immediately access its associated documentation. This allows for a seamless and more convenient experience for the users when navigating to different tabs.

4. Data and Limitations

Tasks, job allocation, deadlines, MoSCoW priority, and documentation entries are all stored locally in Room, so the core task, allocating and tracking work across the team, remains fully usable with no internet connection at all; nothing in the to-do list, timeline, or documentation screens depends on a network call. The one feature that genuinely needs connectivity is the GitHub activity feed (R4), which shows the team's recent commits and implements the required loading, success, empty, and error states, including Retry - so losing connectivity degrades only that one screen rather than the app as a whole.

Tasks reference their assignee through a foreign key to the Member table. If a member is ever removed, for example during testing, or if the team drops a placeholder account, any task assigned to them does not become orphaned or crash the app. The foreign key is configured with ON DELETE SET NULL, so the task's assigneeId is safely cleared and the task simply displays as "Unassigned" on the next load. This was verified by deleting a seeded member directly and confirming their existing tasks reappeared correctly, still intact, just unassigned.

Session state, which screen is showing and which user is logged in, currently lives in memory, in SessionViewModel. A ViewModel survives a device rotation, so rotating mid-session is handled correctly, but it does not survive the process being fully killed, such as when Android reclaims memory in the background or the user force-closes the app - the next launch returns to the Login screen rather than restoring exactly where the user left off. A related, smaller issue is that the logic for deciding whether a task counts as "overdue" is currently implemented separately in four places, the task list's overdue banner, the Timeline's colour coding, the per-task "days remaining" label, and the Home screen's current to-do colouring - rather than behind one shared function everywhere. Each agrees today because they all check the same two conditions (status is not DONE, and the due date has passed), but the duplication is a maintenance risk if the rule ever needs to change.

5. Generative AI Acknowledgement

5.1 Tools:

- Claude (Anthropic, claude.ai)
- GitHub Copilot (suggested commit messages)
- ChatGPT

Date:

September 23, 2026

Group use:

Claude was used to organise our group's rough planning notes into a structured plan. It interpreted the assignment requirements (R1 - R6) and mapped our features to them and suggested out-of-scope features with reasons. It proposed a feature split, with a difficulty-point system to even out workloads across the three of us, along with a feature dependency and build-order table and a dated build schedule. Claude also explained concepts we were unsure about, such as MVVM layer separation, recomposition, building a Gantt chart from Compose layouts and the purpose of a .gitignore. It suggested colour themes with accessibility considerations.

Part of our work it helped produce or revise:

The README's four points (users, core, task, out of scope), the feature allocation and dependency tables, the screen list, the build plan, a draft .gitignore file and handover summaries of our progress

Individual use:

- **Aliyah:**
  Claude explained concepts, APIs and errors (eg. MVVM, StateFlow, Room relationships and migrations, photo picker URI permissions), helped plan features A1-A5 as plain English steps and reviewed code. I instead made and tested all fixes on the emulator by following the broad steps. It also drafted code comments, commit messages and PR descriptions which I reviewed and edited. Additionally, Copilot suggested some commit messages, which I checked before committing.
- **Manaviba:** Claude explained concepts and errors I was unsure about — Room foreign key behaviour and the ON DELETE SET NULL cascade, how ViewModels survive configuration changes like rotation but not process death, and how to structure the task-completion-percentage. When I ran into a blank GitHub screen, it explained debugging approaches as plain English steps (checking Logcat output, comparing against a fresh pull of the branch), which I then worked through myself.
  Twisha:
  ChatGPT explained concepts, such as all the stuff as Aliyah.

I was also a little lost when integrating GitHub into our app, lecture 9 only covers implicit intent which allows us to open the link in an external browser, while still having that feature I wanted to display commits inside of the app so I asked ChatGPT for suggestions, which it recommended Retrofit + REST API + Jetpack Compose.

References List

Adobe. (n.d.). *Color blindness simulator and safe palette tool*. Adobe Color. Retrieved October 18, 2026, from https://color.adobe.com/create/color-accessibility

Android Developers. (n.d.). *ForeignKey*. Retrieved October 18, 2026, from https://developer.android.com/reference/androidx/room/ForeignKey

Android Developers. (2025, July 11). *Material components in Compose*. https://developer.android.com/develop/ui/compose/components

Android Developers. (2026a, September 9). *Migrate your Room database*. https://developer.android.com/training/data-storage/room/migrating-db-versions

Android Developers. (2026b, October 16). *Photo picker*. https://developer.android.com/training/data-storage/shared/photo-picker

Android Developers. (2026c, September 24). *StateFlow and SharedFlow*. https://developer.android.com/kotlin/flow/stateflow-and-sharedflow

Android Developers. (2026d, May 28). *ViewModel overview*. https://developer.android.com/topic/libraries/architecture/viewmodel

Coil Contributors. (n.d.). *Overview*. Coil. Retrieved October 18, 2026, from https://coil-kt.github.io/coil/

GeeksforGeeks. (2025, July 23). *Retrofit with Kotlin coroutine in Android*. https://www.geeksforgeeks.org/kotlin/retrofit-with-kotlin-coroutine-in-android/

Google Cloud. (n.d.). *What is REST API?* Retrieved October 18, 2026, from https://cloud.google.com/discover/what-is-rest-api

JetBrains. (2026, May 18). *How to create RESTful APIs in Kotlin with Ktor*. Ktor Documentation. https://ktor.io/docs/server-create-restful-apis.html

Kar Mahapatra, P. (2024, January 24). Retrofit in Android with Kotlin. *Medium*. https://medium.com/@pritam.karmahapatra/retrofit-in-android-with-kotlin-9af9f66a54a8